

SCSS *Modern* Architecture

ცვლადები, ნესტინგი, იმპორტები და ლოგიკა

Sass

| SCSS ცვლადები (\$)

მონაცემების შენახვა

ცვლადები საშუალებას გვაძლევს ერთ ადგილას განვსაზღვროთ მნიშვნელობები (ფერები, ზომები) და გამოვიყენოთ ისინი მთელ პროექტში.

- ✓ მარტივი განახლება
- ✓ კოდის სისუფთავე

// ფერების განსაზღვრა

```
$bg-dark: #0f172a;
```

```
$primary: #cf649a;
```

// გამოყენება

```
body {
```

```
  background: $bg-dark;
```

```
  color: $primary;
```

```
}
```


| SCSS ნესტინგი (Nesting)

იერარქიული სტრუქტურა

ნესტინგი გვაძლევს საშუალებას დავწეროთ სტილები ისეთივე წყობით, როგორითაც HTML ელემენტებია განლაგებული.

ეს კოდს უფრო **ორგანიზებულს** ხდის.

```
// SCSS Nesting
.header {
  background: white;
  .nav {
    display: flex;
    a {
      text-decoration: none;
      color: blue;
    }
  }
}
```


| ამპერსანდის (&) გამოყენება

სიმბოლო **&** მიუთითებს მშობელ სელექტორზე. ის იდეალურია ფსევდო-კლასებისა და მდგომარეობების სამართავად.

🖱️ Hover და Active ეფექტების სწრაფი მართვა.

```
.nav-link {  
  color: gray;  
  
  &:hover {  
    color: $primary;  
    text-decoration: underline;  
  }  
  
  &:active {  
    transform: translateY(1px);  
  }  
}
```

SCSS იმპორტები

მოდულური არქიტექტურა და ფაილების მართვა

| Partial ფაილები (—)

ქვე-ფაილების შექმნა

SCSS ფაილებს, რომელთა სახელები იწყება `_` ნიშნით (მაგ: `_variables.scss`), ეწოდება "Partials".

ისინი დამოუკიდებელ CSS ფაილებად არ კომპილირდებიან.

ფაილური სტრუქტურა:

```
└─ abstracts/  
  │ └─ _variables.scss  
  │ └─ _mixins.scss  
  └─ components/  
    │ └─ _buttons.scss  
    └─ main.scss
```


| @use დირექტივა

თანამედროვე Sass-ში `@import` ჩაანაცვლა **@use**-მა. ის უფრო უსაფრთხოა და ქმნის "Namespaces".

🛡️ @use თავიდან გვაცილებს ცვლადების კონფლიქტს.

```
// _variables.scss
$color: blue;

// main.scss
@use 'variables';

.button {
  background: variables.$color;
}
```


| SCSS მიქსინები (@mixin)

Reusable Blocks

მიქსინი გვეხმარება CSS კოდის ბლოკების მრავალჯერად გამოყენებაში.

```
@mixin center-flex {  
  display: flex;  
  justify-content: center;  
}
```

```
@mixin theme($bg, $text) {  
  background-color: $bg;  
  color: $text;  
}  
  
.card {  
  @include theme(white, black);  
}
```


| SCSS ფუნქციები (@function)

ფუნქცია გამოიყენება გამოთვლებისთვის და ყოველთვის აბრუნებს კონკრეტულ **მნიშვნელობას**.

იდეალურია ერთეულების კონვერტაციისთვის.

```
@function toRem($px) {  
  @return ($px / 16) * 1rem;  
}  
  
.text-large {  
  font-size: toRem(32); // 2rem  
}
```


| რომელი ინსტრუმენტი გამოვიყენოთ?

ინსტრუმენტი	დანიშნულება	შედეგი
ცვლადი	მუდმივი მნიშვნელობის შენახვა	ერთი მნიშვნელობა
იმპორტი (@use)	მოდულური არქიტექტურა	ფაილების კავშირი
მიქსინი	სტილების ბლოკის რეალიზაცია	CSS კოდის ბლოკი
ფუნქცია	ლოგიკური გამოთვლები	გამოთვლილი მნიშვნელობა

| შეჯამება და დასკვნა



Clean Code

ცვლადები და ნესტინგი კოდს ხდის უფრო წაკითხვადს და ადვილად სამართავს.



Modular

Partials და @use საშუალებას გვაძლევს დავყოთ პროექტი ლოგიკურ მოდულებად.



Logic

მიქსინები და ფუნქციები პროგრამულ შესაძლებლობებს მატებს სტატიკურ CSS-ს.

SCSS არის CSS-ის მომავალი, რომელიც დეველოპმენტს ხდის ეფექტურს.